FULL STACK DEVELOPMENT WITH MERN

PROJECT DOCUMENTATION

1. INTRODUCTION

Project Title: Medical Diagnosis Center by ServiceNow Service Portal

Team Name: TeamNow

Team ID: 23891A05C2

Team Member: Shaik Sameera

Role: Application Development, Service Portal Configuration, Widget Development, Workflow Configuration, and Testing

Project Description

The Medical Diagnosis Center is a web-based application developed using the ServiceNow platform and Service Portal. It is designed to simplify the management of patients, diagnostic tests, appointments, and laboratory reports through a centralized system.

The application provides a user-friendly interface that helps users access diagnostic test information, book appointments, track appointment status, and view available laboratory reports. It aims to improve the efficiency and organization of diagnostic center operations.

2. PROJECT OVERVIEW

Purpose

The primary purpose of this project is to digitize and streamline the daily activities of a medical diagnostic center. Traditional manual processes can make patient registration, test booking, appointment tracking, and report management difficult to organize.

This project provides a centralized platform to manage these activities efficiently. It aims to reduce manual effort, improve data organization, and provide a convenient experience for patients and staff.

Features

1. Test Catalog: Displays available diagnostic tests and their information through the Service Portal.

2. Patient Management: Supports the management of patient information through the configured application tables and forms.

3. Appointment Booking: Provides an interface for users to book diagnostic tests and manage appointment details.

4. My Appointments: Displays appointment records and their statuses, such as Pending, Confirmed, Completed, and Cancelled, according to the implemented configuration.

5. Laboratory Report Management: Provides a dedicated interface for accessing laboratory report information.

6. Service Portal Interface: Provides a centralized and user-friendly interface for interacting with the application.

7. Centralized Data Management: Stores and organizes application information using ServiceNow tables.

8. Workflow Automation: Uses configured ServiceNow workflows or flows to automate selected application processes.

3. ARCHITECTURE

The application follows a platform-based architecture using ServiceNow. The Service Portal provides the user interface, widgets handle portal interactions, server-side scripts process application logic, and ServiceNow tables store the required records.

Frontend Architecture

- The frontend is developed using ServiceNow Service Portal pages and widgets. These components provide the interface through which users can view diagnostic tests, manage appointments, and access laboratory report information.
- The Test Catalog page displays available diagnostic tests. The My Appointments interface helps users view and manage their appointment records. The Lab Reports page provides access to report-related information.
- Service Portal widgets connect the user interface with server-side logic to retrieve and process application data.

Backend Architecture

- The backend functionality is implemented using ServiceNow server-side scripting and platform capabilities. Server-side scripts process requests, retrieve records, and perform application operations according to the configured business logic.
- ServiceNow tables store patient, appointment, diagnostic test, and report information. Configured flows can automate selected tasks and update records based on defined conditions.

Database Architecture

The project uses the ServiceNow platform's database and application tables instead of a separate MongoDB database.

The primary application tables are:

1. Patient Table: Stores patient-related information required by the application.

2. Appointment Table: Stores appointment details, including the associated patient, selected test, appointment information, and status fields as configured.

3. Diagnostic Test Table: Stores information about the diagnostic tests available through the application.

4. Report Table: Stores laboratory report information associated with the diagnostic process.

These tables support data storage and retrieval for the application's portal pages and widgets. Relationships between records depend on the configured reference fields and application logic.

Architecture Flow

User → Service Portal Pages → Service Portal Widgets → Server-Side Scripts and Business Logic → ServiceNow Tables → Response Displayed on the Portal

4. SETUP INSTRUCTIONS

Prerequisites

The following are required to access and work with the application:

- A modern web browser such as Google Chrome.
- An active ServiceNow instance with appropriate access permissions.
- Access to the configured Medical Diagnosis Center application.
- ServiceNow development access for modifying tables, widgets, scripts, and flows.
- The Service Portal URL for accessing the application.

Node.js, Express.js, and MongoDB are not required for the current implementation because the project uses the ServiceNow platform.

Installation and Configuration

Step 1: Access ServiceNow

Log in to the configured ServiceNow instance using authorized credentials.

Step 2: Open the Application

Navigate to the application containing the Medical Diagnosis Center components.

Step 3: Verify Application Tables

Ensure that the required Patient, Appointment, Diagnostic Test, and Report tables are configured and accessible.

Step 4: Verify Service Portal Pages

Check that the required portal pages, including the Test Catalog and Lab Reports pages, are available.

Step 5: Verify Widgets

Confirm that the configured widgets are associated with the correct portal pages and use the appropriate table names and server-side logic.

Step 6: Verify Scripts and Flows

Check that the required server-side scripts and configured flows are active and operating as expected.

Step 7: Open the Portal

Access the Service Portal using the configured portal URL and verify that the pages and application features are displayed correctly.

5. FOLDER STRUCTURE

Unlike a traditional MERN application, this project is organized using ServiceNow application components rather than separate client and server folders.

The logical structure of the application is as follows:

Medical Diagnosis Center Application

Service Portal Pages

1. Test Catalog
2. Lab Reports
3. My Appointments

Service Portal Widgets

4. Test Catalog Widget
5. Appointment Management Widget
6. Laboratory Report Widget

Application Tables

7. Patient Table
8. Appointment Table
9. Diagnostic Test Table
10. Report Table

Server-Side Components

11. Widget server scripts
12. Record retrieval and update logic

13. Configured business logic

Automation

- Configured ServiceNow flows and workflow components

The names above describe the logical organization of the application. Actual component names may differ according to the implementation in the ServiceNow instance.

6. RUNNING THE APPLICATION

The application runs on the configured ServiceNow instance. It does not require separate frontend and backend servers to be started using npm commands.

Steps to Run

1. Open the web browser.
2. Navigate to the configured ServiceNow instance.
3. Log in using authorized credentials.
4. Open the Medical Diagnosis Center Service Portal URL.
5. Navigate to the Test Catalog page.
6. Verify the available diagnostic tests.
7. Open the appointment interface to view or book appointments.
8. Navigate to My Appointments to review appointment records and statuses.
9. Open the Lab Reports page to view the available report information.

The application depends on the availability of the ServiceNow instance and the correct configuration of its portal pages, widgets, tables, and scripts.

7. APPLICATION COMPONENT DOCUMENTATION

- Since the project uses ServiceNow rather than a custom Node.js and Express.js backend, it does not necessarily expose a set of custom REST API endpoints.
- The main application interactions are implemented through Service Portal widgets, server-side scripts, and ServiceNow table operations.

Component 1: Test Catalog

Purpose: Displays diagnostic tests available in the application.

Input: User request to open or view the test catalog.

Processing: The configured widget retrieves diagnostic test information from the relevant ServiceNow table.

Output: The available test information is displayed on the portal page.

Component 2: Appointment Management

Purpose: Supports appointment-related operations.

Input: Appointment details submitted through the portal interface.

Processing: The configured widget and server-side logic validate and process the submitted information according to the implemented rules.

Output: Appointment information is stored or displayed according to the result of the operation.

Component 3: My Appointments

Purpose: Displays appointment records and their current statuses.

Input: A request to view appointment information.

Processing: The widget retrieves relevant appointment records using the configured server-side logic.

Output: Appointment records are displayed with the status information available in the application.

Component 4: Lab Reports

Purpose: Provides access to laboratory report information.

Input: A request to view available report records.

Processing: The configured widget retrieves report information from the relevant ServiceNow table.

Output: The available report information is displayed on the portal page.

Request and Response Handling

Service Portal widgets can use client-side and server-side scripts to exchange information. The server-side script processes the request and interacts with ServiceNow records. The resulting data is returned to the widget and displayed to the user.

Custom REST endpoints, request parameters, and response formats should be documented separately if they have been implemented in the application.

8. AUTHENTICATION AND AUTHORIZATION

The application uses the authentication and access-control capabilities available on the ServiceNow platform.

Authentication

Users access the application through the configured ServiceNow instance. Authentication is handled according to the instance's login and security configuration.

Authorization

Access to application data and functionality should be controlled through the relevant ServiceNow roles, table permissions, and access control rules configured for the application.

Data Security

Patient and laboratory information should only be accessible to authorized users. Access permissions should be configured and tested to ensure that users can access only the records and functions permitted by their roles.

The project does not require a separate JWT authentication system unless one has been specifically implemented.

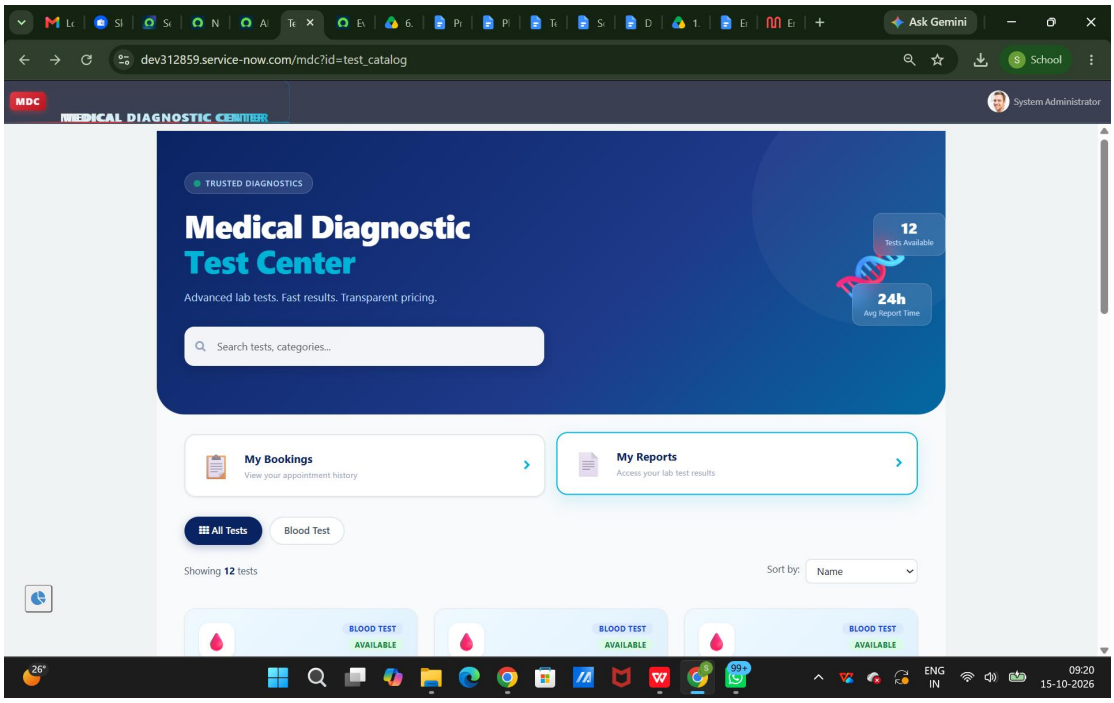
9. USER INTERFACE

The application provides a Service Portal interface for accessing diagnostic center services.

The following screenshots should be added to demonstrate the implemented features.

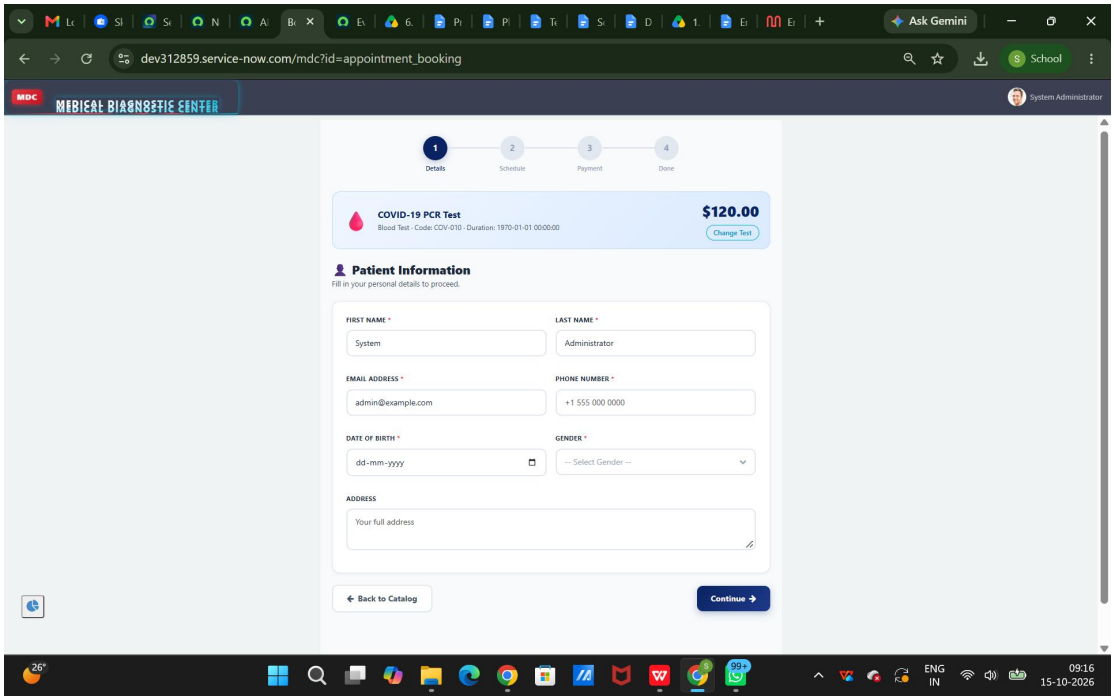
9.1 Test Catalog Page

Description: This page displays the diagnostic tests available through the application and allows users to explore the available options.



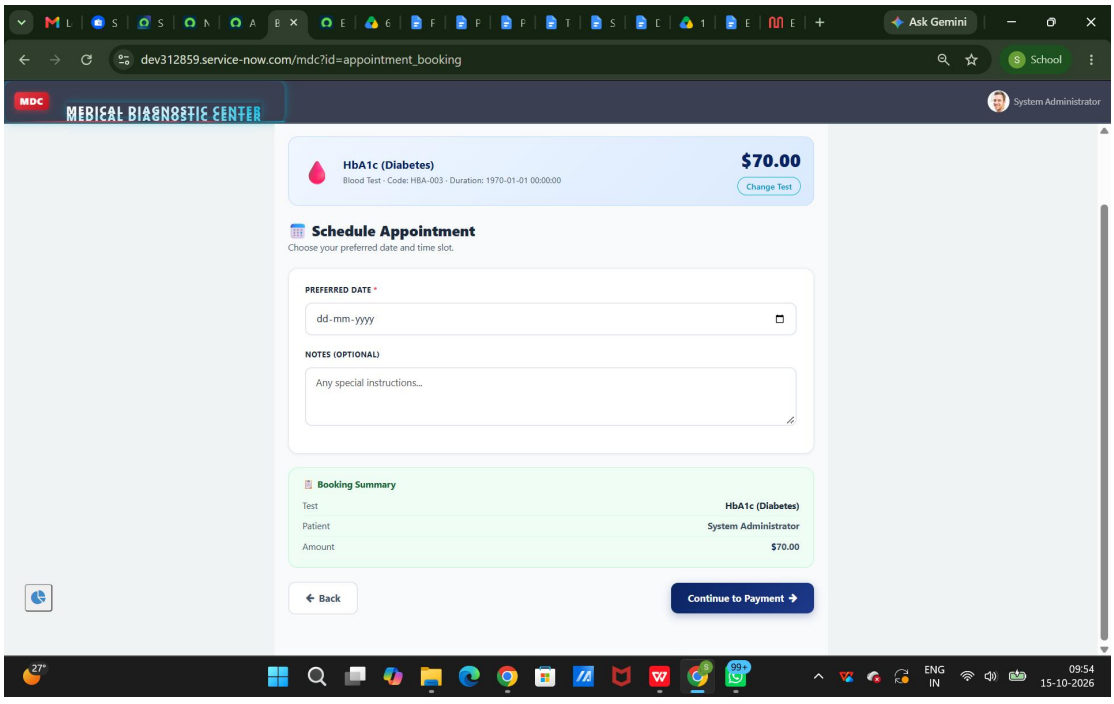
9.2 Patient Registration Page

Description: This interface supports the collection and management of patient information according to the configured registration process.



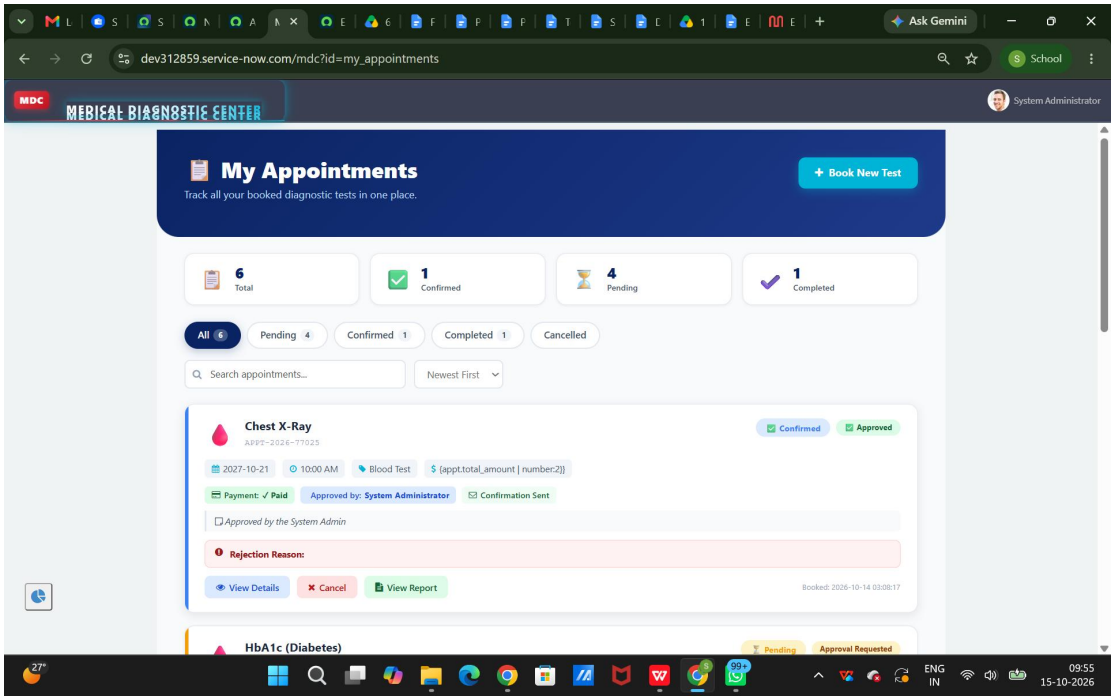
9.3 Appointment Booking Page

Description: This interface allows users to enter the required appointment information and submit an appointment request.



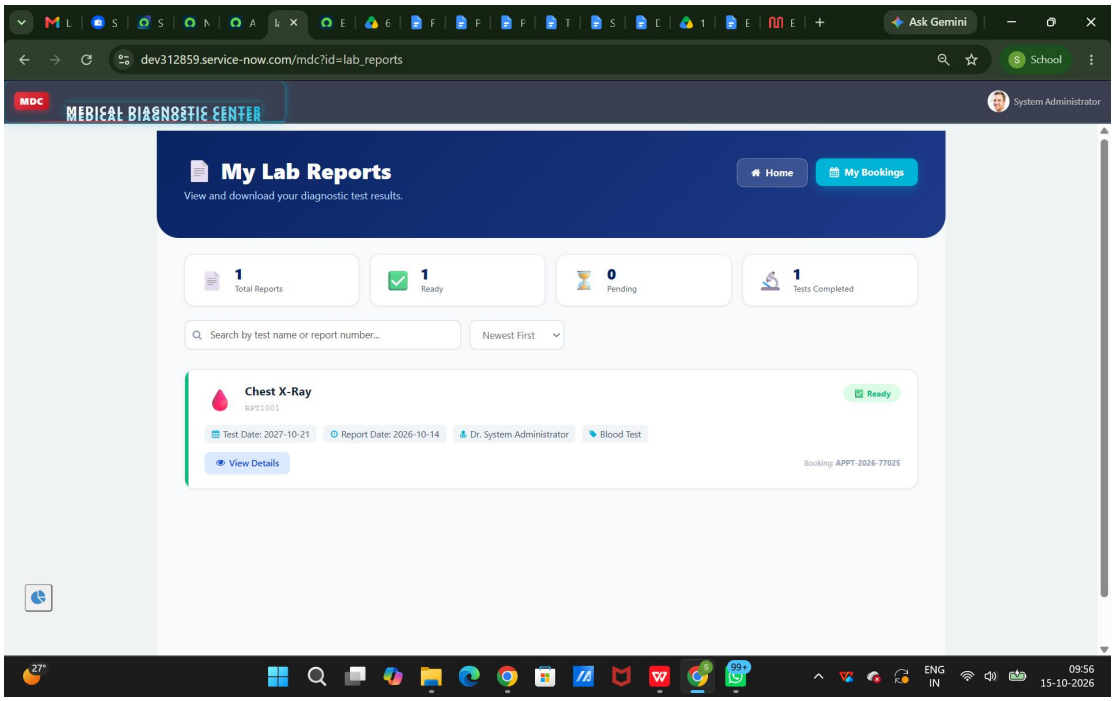
9.4 My Appointments Page

Description: This page displays appointment records and their statuses. The available status filters and actions depend on the implemented widget configuration.



9.5 Lab Reports Page

Description: This page provides an interface for viewing available laboratory report information.



10. TESTING

Testing is necessary to verify that the application operates correctly and that its pages, widgets, and data interactions work as expected.

Testing Strategy

The application should be tested using the following approaches:

1. Functional Testing: Verify that each implemented feature performs its intended operation.

2. Interface Testing: Check that portal pages, forms, buttons, filters, and navigation elements work correctly.

3. Data Validation Testing: Verify that required fields are handled correctly and that records are stored or retrieved as expected.

4. Workflow Testing: Check the execution of configured flows and verify the expected record updates.

5. Access-Control Testing: Verify that users can access only the records and functions permitted by their assigned roles.

Test Cases

Test Case 1: Open Test Catalog

Expected Result: The Test Catalog page opens and displays the available diagnostic test information.

Actual Result: To be recorded after testing.

Test Case 2: Patient Registration

Expected Result: The registration process accepts valid information and handles invalid or missing required fields appropriately.

Actual Result: To be recorded after testing.

Test Case 3: Appointment Booking

Expected Result: Valid appointment information is processed according to the configured application logic.

Actual Result: To be recorded after testing.

Test Case 4: View My Appointments

Expected Result: Relevant appointment records and their statuses are displayed correctly.

Actual Result: To be recorded after testing.

Test Case 5: View Laboratory Reports

Expected Result: Available report information is displayed according to the configured access permissions.

Actual Result: To be recorded after testing.

Test Case 6: Workflow Execution

Expected Result: Configured flows execute under the defined conditions and perform the expected actions.

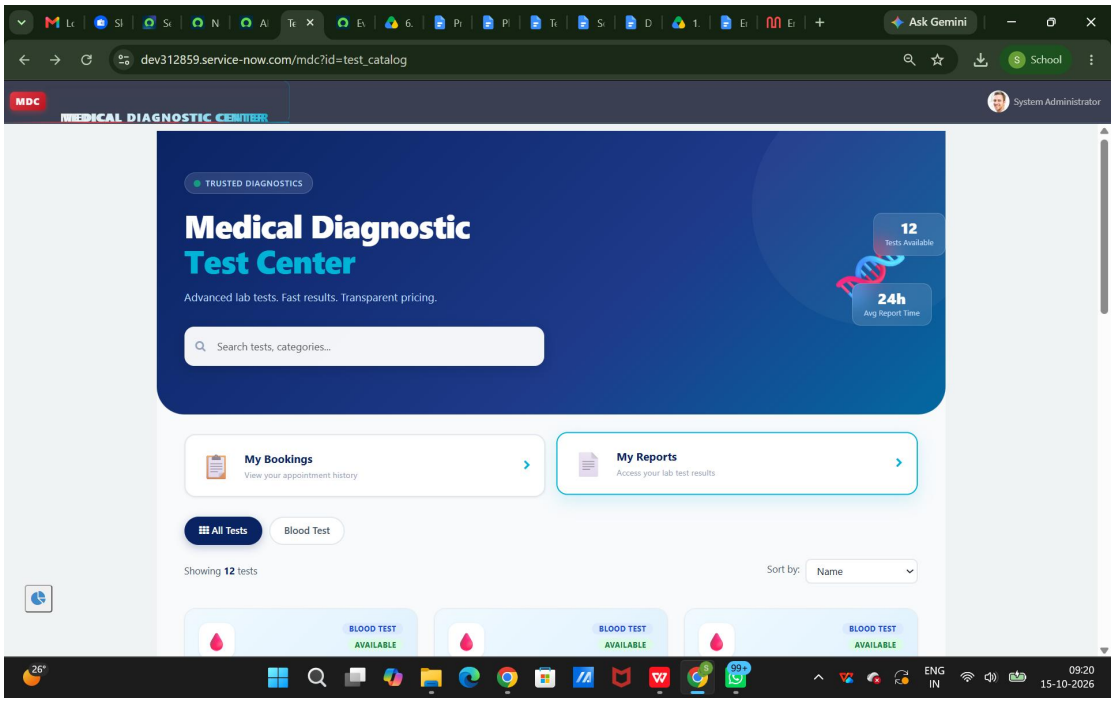
Actual Result: To be recorded after testing.

Testing results must be updated based on actual execution in the ServiceNow instance. A test case should be marked as Passed only after the expected result has been verified.

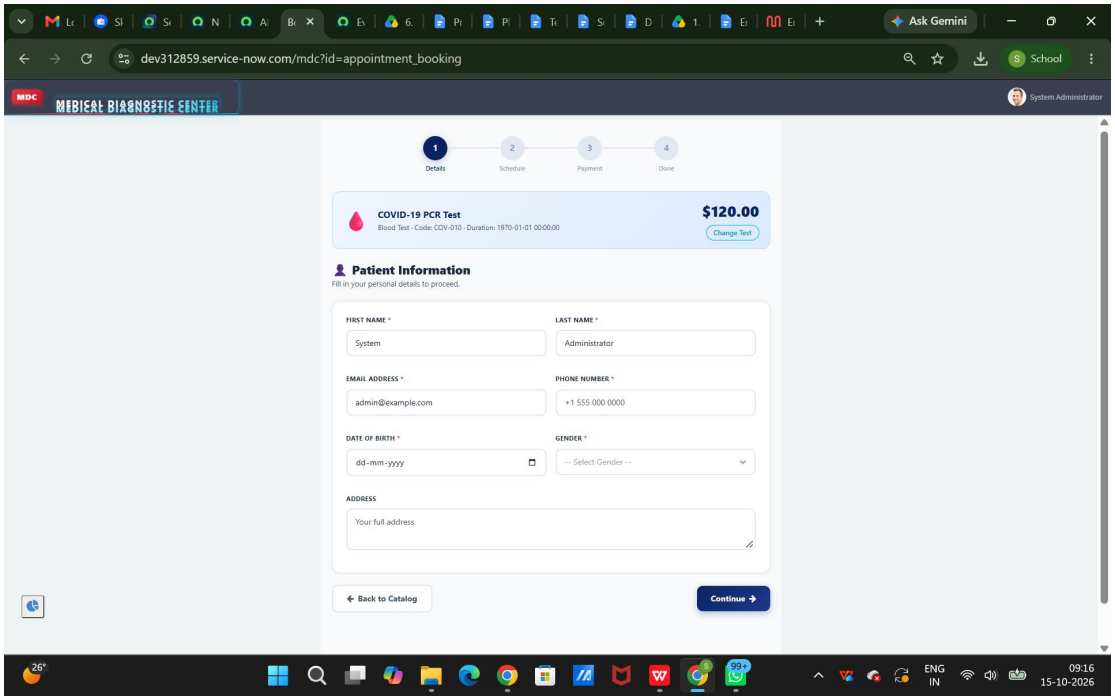
11. SCREENSHOTS OR DEMO

Screenshots of the working application should be included to demonstrate the implementation and user interface.

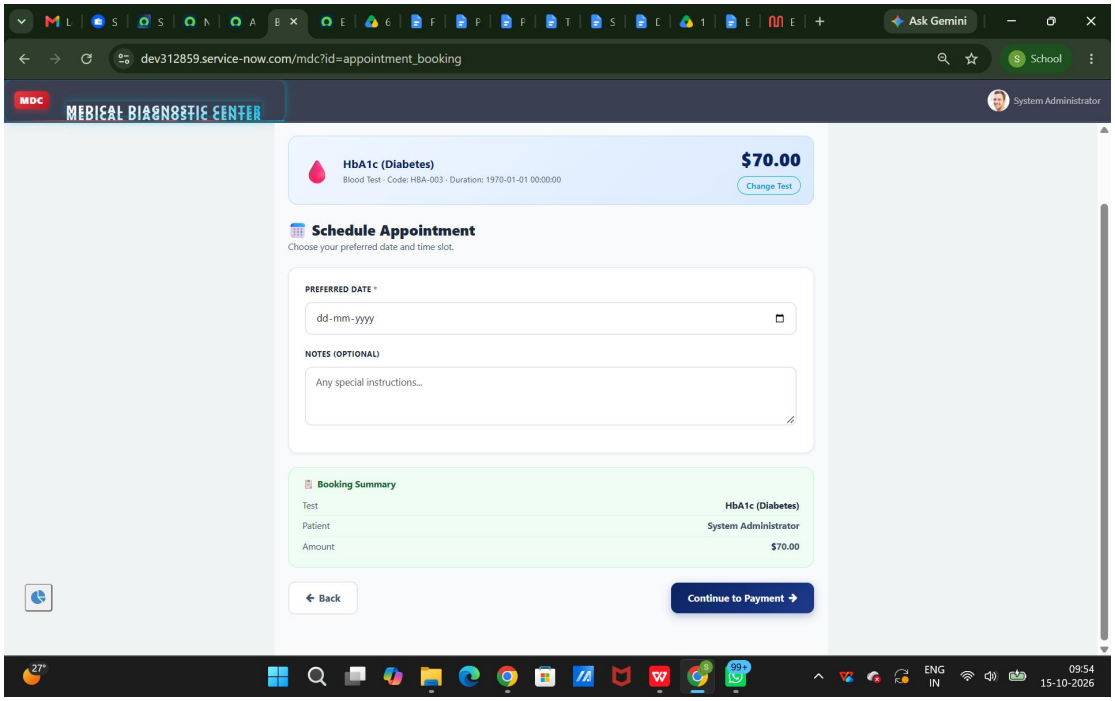
The recommended screenshots are:



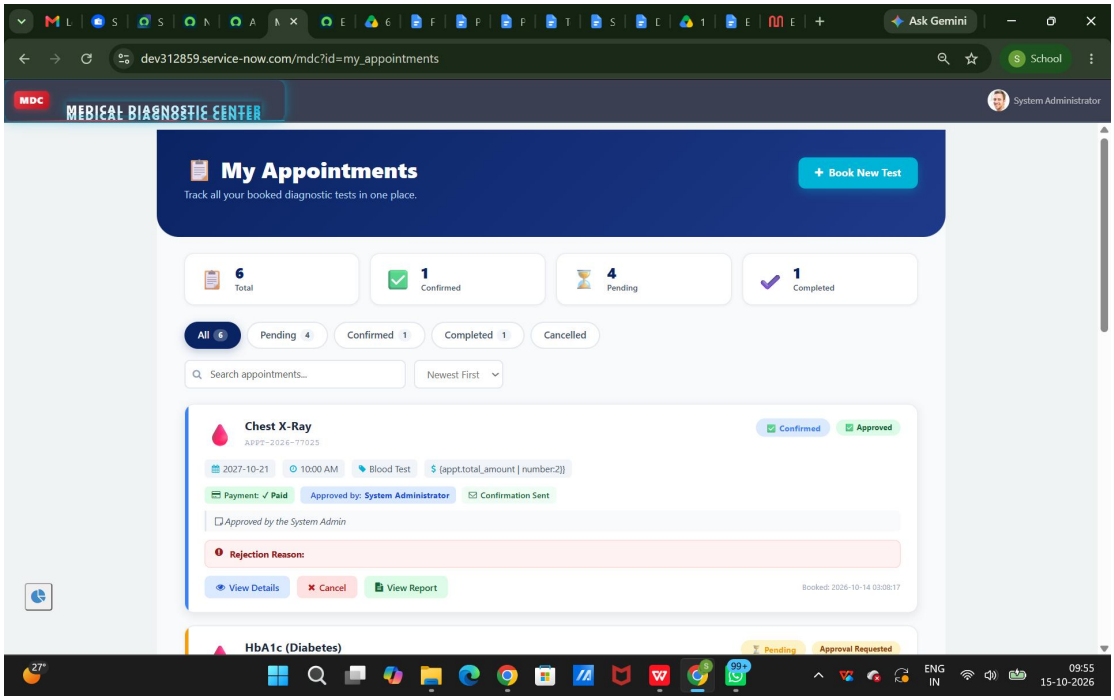
Patient Registration interface.



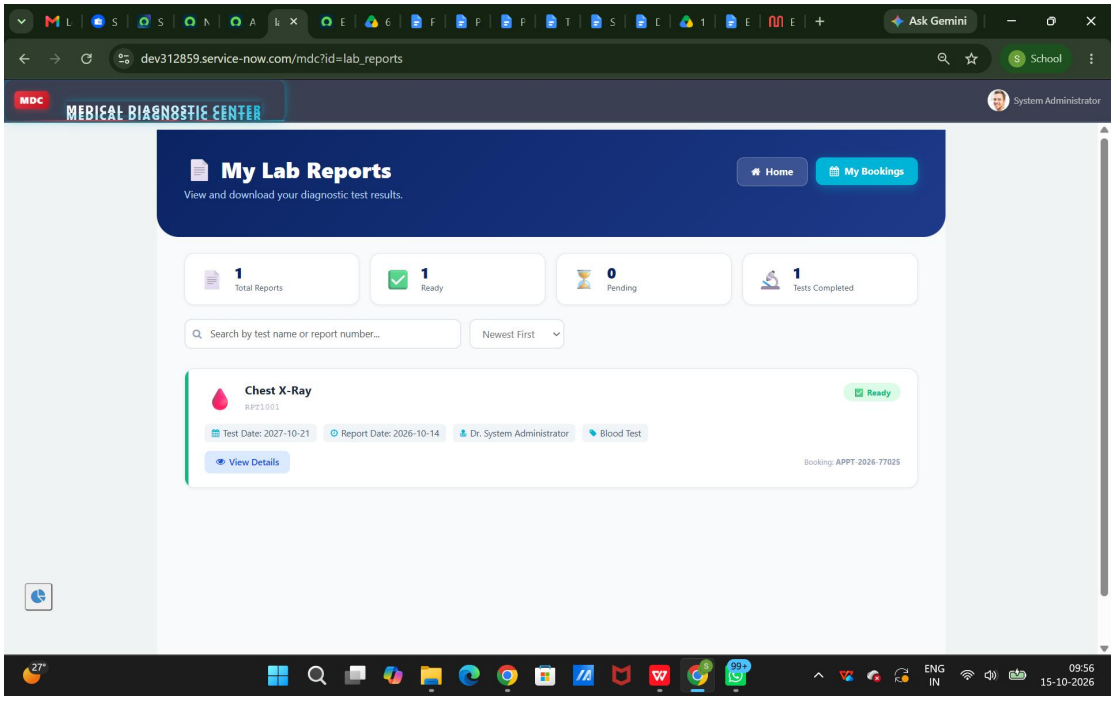
Appointment Booking interface.



My Appointments page.



Lab Reports page.



12. KNOWN ISSUES

The following areas should be checked before final submission:

- Incorrect table names or field mappings may prevent widgets from retrieving the expected records.
- Incorrect widget configuration may cause portal pages to display incomplete or unexpected information.

- Missing or incorrect access permissions may prevent authorized users from viewing or updating records.
- Incorrect flow conditions may prevent expected automation from executing.
- Missing required-field validation may allow incomplete records to be submitted.

These are potential issues to verify, not confirmed defects. The final documentation should list only the issues actually observed during testing. If no issues remain in the tested scope, state that no known issues were identified during testing.

13. FUTURE ENHANCEMENTS

The project can be extended with additional features to improve functionality and usability.

1. Automated Notifications: Introduce email notifications for appointment confirmations, status changes, and report availability.

2. Advanced Appointment Scheduling: Add improved slot management and conflict prevention for appointment booking.

3. Report Downloads: Provide secure report downloads for authorized users where supported by the configured application.

4. Dashboard and Analytics: Develop dashboards to display appointment counts, diagnostic test usage, pending reports, and other operational metrics.

5. Enhanced Role-Based Access: Improve access control for patients, receptionists, laboratory technicians, and administrators.

6. Payment and Invoice Management: Integrate payment and billing features if required by the diagnostic center.

7. Improved User Experience: Enhance portal navigation, accessibility, responsiveness, and validation messages.

8. Audit and Monitoring: Improve activity tracking and monitoring of important record updates.

These enhancements can be considered for future versions based on user requirements and platform capabilities.

CONCLUSION

- The Medical Diagnosis Center by ServiceNow Service Portal project demonstrates how a centralized application can support the management of diagnostic tests, patient information, appointments, and laboratory reports.
- By using ServiceNow Service Portal pages, widgets, application tables, server-side logic, and configured automation, the application provides a structured approach to managing diagnostic center operations.

- The project establishes a foundation for improving data organization, reducing manual effort, and providing a convenient interface for users. Further testing and future enhancements can improve its reliability, security, and overall functionality.